arXiv:2611.09474v1 [cs.DC] 6 Nov 2026

NEBULA: Distributed Position-Independent Caching for LLM Serving

Yifei Liu[1*†] Junbin Pan[1*] Junhao Hu[3] Yang Liu[2] Junyi Chen[2]
Juntong Wu[3] Chen Chen[2] Haiyuan Zhao[1] Xiaoxu Chen[1] Weihang Chen[1]
[1]*Xiaohongshu* [2]*Shanghai Jiao Tong University* [3]*Peking University*

Abstract

Modern LLM workflows, such as retrieval-augmented generation (RAG) and LLM agents, build long prompts from multiple independent text segments, making prefill a major source of computation. Reusing KV caches is therefore important for reducing serving costs and latency: distributed caching systems spread KV cache across a cluster to increase aggregate capacity, while position-independent caching (PIC) enables reuse of recurring segments under different prefixes. However, existing systems distribute KV caches but still rely on prefix caching (PC), while existing PIC methods enable position-independent reuse without cluster-wide resource management.

We present NEBULA, the first distributed LLM serving framework to treat PIC segments as first-class citizens, enabling remote PIC reuse and parallel prefill. To track PIC's fine-grained, asynchronous state changes, NEBULA combines optimistic accounting with event-driven updates to provide a timely and truthful cluster view for cache management and prefill scheduling. To accommodate diverse reuse patterns across workloads, NEBULA leverages cluster-wide ghost-hit feedback to automatically tune the allocation of shared cache capacity between PC and PIC. Finally, NEBULA incorporates queueing, computation, and KV transfer costs into a unified prefill completion time model to guide elastic parallelism adjustment and locality-aware compute placement. Across four models and four real-world RAG and agentic workloads, NEBULA reduces median TTFT by an average of 61.4% compared with the strongest existing baseline and recovers an average of 76.1% of the tokens missed by prefix caching.

1 Introduction

Large language model (LLM) serving is expanding beyond single-turn chat to support retrieval-augmented generation (RAG) and long-horizon agents. RAG inserts retrieved documents into prompt templates [2, 4, 12, 28], while long-horizon agents extend their context with tool outputs, memory and skill files [7, 21, 32]. Prefilling these growing contexts requires substantial GPU computation and increases time-to-first-token (TTFT). Reusing KV caches to avoid repeated prefill computation is therefore important for lowering serving costs and improving responsiveness.

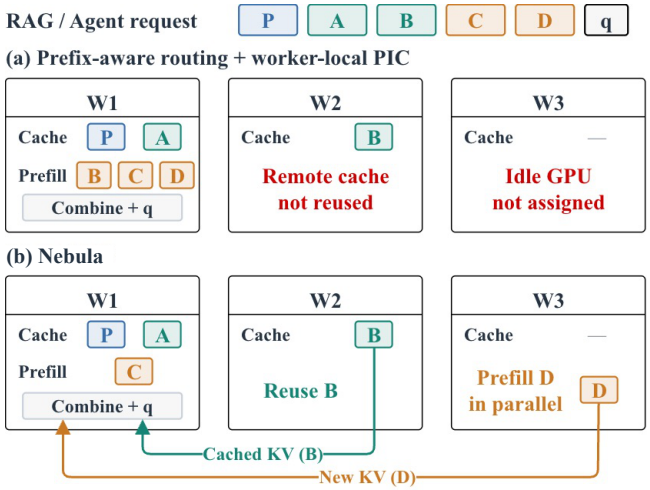


Figure 1: Distributed PIC for a single request. Both plans start with the same cache placement and worker loads, with PIC segments A and B cached on different workers. (a) Worker-local PIC reuses A but leaves remote B and an idle worker unused. (b) NEBULA reuses B across workers and computes C and D on separate workers while preserving prefix reuse. W1 combines the KV cache and processes query segment q.

Distributed inference systems extend KV caching across the cluster. Mooncake [22], LMCache [11], and MemServe [5] support cross-node indexing, storage, and transfer, allowing requests to reuse KV generated on other nodes and benefit from aggregate cache capacity. Yet their distributed reuse still relies on strict prefix caching (PC): only content

*Both authors contributed equally to this work.
†Yifei Liu is the corresponding author (liuyifei5@xiaohongshu.com).

that matches continuously from the beginning of a request can be reused. PC therefore cannot reuse a previously seen document or tool result independently when its preceding context changes.

In parallel, position-independent caching (PIC) [6, 9, 13, 14, 26, 27, 31] expands the scope of reuse. Methods such as CacheBlend [31] and EPIC [6] cache semantically independent segments and *combine* their KV under a different prefix with correction or recomputation as needed. This matches the recombination of retrieved documents in RAG and the reuse of shared content in different agent contexts. Across six real-world RAG and agent workloads, our reuse breakdown shows that PIC can recover approximately 72.1% of the tokens missed by PC on average (§3.1).

However, simply combining existing PIC algorithms with distributed inference systems does not realize the full capabilities of distributed PIC. Routers still match caches by prefix and cannot locate cached PIC segments across nodes; schedulers still treat each request as monolithic and cannot parallelize prefill of its cold segments across workers. Figure 1 illustrates the resulting missed opportunities for remote cache reuse and parallel prefill.

We present NEBULA, the first distributed LLM serving framework to treat PIC segments as first-class citizens. NEBULA enables remote PIC reuse and parallel prefill of uncached segments across workers while preserving PC semantics. Through a unified backend interface, NEBULA supports multiple PIC algorithms and decouples algorithm-specific KV processing from cluster-wide resource management. Building on this framework, we further introduce three designs, each addressing a distinct challenge posed by distributed PIC.

C1: Keeping the cluster view timely and truthful through optimistic, event-driven view synchronization. Distributed PIC can break a single request into hundreds of scheduling units whose states change asynchronously across workers, while routing decisions and actual execution are separated in time. Inferring cache state solely from scheduling decisions may report invalid cache hits, while relying solely on execution feedback may lead to redundant computation. NEBULA combines these complementary sources: the router immediately records expected cache locations and compute load upon dispatch, and workers confirm or correct these records through cache residency, eviction, and task completion events. This allows NEBULA to maintain an accurate and timely cluster view, providing a common basis for cache management and prefill scheduling (§5).

C2: Balancing PC and PIC allocation through shared, self-tuning cache management. Workloads differ in their PC and PIC reuse patterns: RAG documents recur under different prefixes, while successive agent interactions extend the same prefix. Uncontrolled competition between PC and PIC for shared cache space can cause cache pollution, while fixed quotas can strand cache capacity. NEBULA dynamically adjusts the target PC/PIC capacity ratio using cluster-wide ghost-hit feedback, which captures reuse demand after eviction. Capacity allocation thus adapts to the actual reuse patterns observed across the cluster (§6).

C3: Balancing compute and transfer through elastic, locality-aware prefill scheduling. A distributed PIC request must both fetch cached KV and compute uncached segments. Placing computation on cache-affine nodes may cause load imbalance, while placing it on idle nodes may increase network traffic. Optimizing cache locality or load balance alone does not necessarily minimize the prefill completion time of a PIC request. NEBULA incorporates queueing, computation, and KV transfer costs into a unified prefill completion time estimate and uses a greedy search to select the best candidate execution plan. The prefill plans therefore dynamically adjust parallelism and compute placement as load and cache distribution change, accounting for both available compute resources and data locality (§7).

We implement NEBULA on SGLang [33] and Dynamo [18], with support for PIC methods including CacheBlend [31], A^3 [34], EPIC [6] and HYPIC [10]. We evaluate NEBULA with four diverse models on four real-world RAG and agentic workloads. NEBULA reduces median TTFT by an average of 61.4% compared with the strongest existing baseline and recovers an average of 76.1% of the tokens missed by prefix caching. NEBULA's source code will be released at https://github.com/redai-studio/Nebula.

2 Background

2.1 Prefix Caching at Scale

Prefix Caching. To reduce prefill computation and TTFT for long contexts, mainstream LLM serving systems, such as vLLM [8] and SGLang [33], reuse KV cache across requests with prefix caching (PC). Under causal attention, each token's KV state depends on that token and its preceding context. PC therefore requires strict prefix matching: when a new request shares its first n tokens with an earlier request, it can reuse the corresponding KV cache directly with exact numerical equivalence. Once a prefix mismatch occurs, the KV cache from that point onward becomes immediately unusable.

Distributed Caching Systems. In a serving cluster with multiple LLM instances, reusable KV cache may reside on different nodes and storage tiers, requiring efficient lookup and access during request execution. Mooncake [22], LMCache [11], and MemServe [5] support cluster-wide reuse through cache indexing, tiered storage, and KV transfers across nodes, with Mooncake and MemServe further considering cache location and compute load during request scheduling. TokenLake [29] extends prefix cache management to the segment level, while segment reuse still requires prefix matching.

Category	Workload	PC%	PIC%	NPHR%	PMRR%
RAG	ChatRAG-doc2dial [4, 12]	2.63	99.68	97.06	**99.7**
RAG	ChatRAG-QuAC [2, 12]	3.15	87.50	84.36	**87.1**
RAG	RAGPulse [28]	31.53	72.01	40.48	**59.1**
Agent	nebius-SWE-agent [1]	95.26	97.49	2.23	**47.0**
Agent	τ-bench-Retail [32]	92.62	96.69	4.07	**55.1**
Agent	ChatDev [21]	77.59	96.57	18.98	**84.7**

Table 1: Theoretical cache hit analysis of PC and PIC on RAG and agent workloads.

2.2 Efficient Position-Independent Caching

Position-Independent Caching. Position-independent caching (PIC) relaxes strict prefix matching and allows semantically independent segments to be reused at arbitrary positions. The cached KV still depends on the segment's original position and preceding context, so direct concatenation introduces numerical deviations. PIC methods typically use recomputation to reduce these deviations: CacheBlend [31] recomputes tokens with the largest KV deviations; A^3 [34] identifies key tokens for recomputation from attention distributions; EPIC [6] uses attention sink patterns to statically select the first k tokens of each chunk for recomputation. HYPIC [10] further enables PIC for linear attention through transition matrices.

System Gap. However, existing distributed caching systems are built on prefix caching, and none provides complete distributed support for PIC. CacheBlend implements PIC on top of LMCache but does not integrate it into distributed cache management. MEPIC [26] and MiniPIC [20] explore practical PIC implementations but do not consider distributed support. HYPIC is the closest work to ours, but it uses distributed computation only to accelerate cold prefill and does not address the new challenges that PIC introduces for distributed caching systems (§4.3).

3 Analysis

As discussed in §2, PIC can reuse non-prefix content that PC cannot, yet no existing system fully supports distributed PIC. Before diving into NEBULA design (§4), we first examine two questions: (i) How much additional reuse can PIC provide on top of prefix caching for RAG and agent workloads (§3.1)? (ii) Is deploying single-node PIC within existing distributed prefix caching systems sufficient to exploit the cluster's cache and compute resources (§3.2)?

3.1 The Opportunity for PIC

PIC Benefit on Prefix Caching. Although PC yields low hit rates on RAG workloads, existing production systems already achieve cache hit rates above 90% on agent workloads. A meaningful assessment of PIC's incremental benefits for RAG and agent workloads must therefore account for what existing systems already achieve. Let h_{PC} and h_{PIC} denote the fractions of a request's input tokens reusable through prefix caching and position-independent caching, respectively. In this theoretical matching analysis, a prefix is a special case of a segment at the beginning of the sequence, so $h_{\text{PIC}} \geq h_{\text{PC}}$. We use two metrics to quantify PIC's additional benefit over prefix caching:

- Non-prefix hit rate (NPHR) $= h_{\text{PIC}} - h_{\text{PC}}$: the fraction of input tokens that miss prefix matching but hit position-independent matching, or the absolute increase in hit rate provided by PIC.

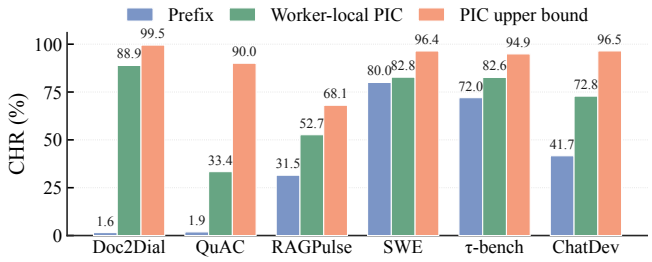
- Prefix-miss recovery rate (PMRR) $= \text{NPHR} \,/\, (1 - h_{\text{PC}})$: the fraction of prefix-missed tokens that PIC can reuse, directly reflecting how much computation can be further reduced after prefix caching has been fully exploited.

We select six public workloads spanning RAG and agent scenarios. For RAG, ChatRAG-doc2dial [4, 12] and ChatRAG-QuAC [2, 12] are conversational RAG workloads, and RAGPulse [28] is a public RAG serving trace. For agents, nebius-SWE-agent [1] is a coding agent trace, τ-bench-Retail [32] is a tool-use agent trace, and ChatDev [21] is a multi-agent software collaboration trace.

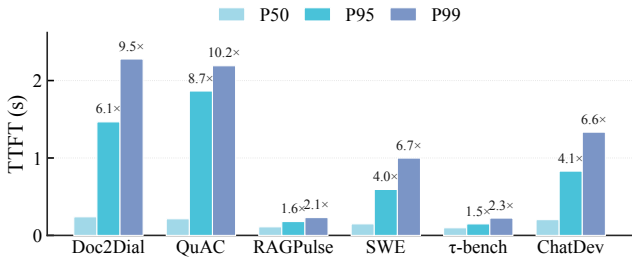
As shown in Table 1, RAG and agent workloads exhibit markedly different reuse patterns. RAG requests concatenate independently retrieved texts after a system prompt, often leaving repeated content outside the shared prefix. ChatRAG-doc2dial and ChatRAG-QuAC have prefix hit rates of only about 3%, yet their NPHR reaches 97% and 84%, respectively, with most hits coming from non-prefix reuse. Even RAGPulse, with a higher prefix hit rate of 31.5%, has a PMRR of 59.1%.

Agent requests grow over successive interactions, giving tool-use and coding workloads high prefix hit rates: 92.6% for τ-bench-Retail and 95.3% for nebius-SWE-agent, with NPHR of only 2% to 4%. Yet the dominance of prefix reuse does not mean that PIC has no value: for the less than 10% of input tokens that miss the prefix cache, PMRR is still 55.1% and 47.0% in these two workloads, respectively. A role-level breakdown further shows that environment observations account for 85.4% of the non-prefix reuse in nebius-SWE-agent, while tool-returned database records account for 89.5% in τ-bench-Retail. **This means that even with such high prefix hit rates, nearly half of the remaining computation can still be saved by PIC.** In the multi-agent workload ChatDev, content shared across agents appears at different positions, also creating considerable PIC reuse opportunities with a PMRR of 84.7%.

Despite their different reuse patterns, both RAG and agent workloads can save a substantial fraction of prefill computation by adding PIC to existing PC.



(a) Cache hit ratio.



(b) Tail TTFT of worker-local PIC.

Figure 2: Limitations of deploying PIC-enabled workers behind a prefix-cache-aware router. (a) Worker-local PIC improves cache hit ratio (CHR) over PC but remains below the ideal PIC upper bound. (b) Worker-local PIC exhibits substantial tail TTFT across the same workloads.

3.2 From PIC to Distributed PIC

The Gap between PIC and Distributed PIC. To assess the gap between the existing system and the ideal state, we first construct a direct-composition strawman by deploying PIC-enabled workers behind Dynamo, an existing prefix-cache-aware router. On the one hand, this setup lacks **remote cache reuse**: it cannot locate cached PIC segments on other nodes, so it may recompute content that could instead be fetched over the network for reuse. On the other hand, this setup lacks **parallel prefill**: it cannot offload independently computable PIC segments to idle nodes, so it may leave available cluster compute resources unused and suffer from high TTFT.

We deploy the worker-local PIC strawman in a cluster with seven prefill workers and one decode worker, and replay the six workloads at 8 requests/s in their original order from a cold cache. Figure 2 (a) shows that worker-local PIC improves CHR over PC, yet remains 10.6–56.7 percentage points below the ideal PIC upper bound across the six workloads. The gap is largest on ChatRAG-QuAC, where worker-local PIC achieves only 33.4% CHR against an upper bound of 90.0%. Figure 2 (b) further shows substantial tail TTFT: P99 is 2.1–10.2× P50, reaching 9.5× on ChatRAG-doc2dial and 10.2× on ChatRAG-QuAC. Because each request remains bound to one worker, the strawman cannot use idle workers to accelerate the prefill of its uncached PIC segments.

These observations motivate a distributed PIC system that can fully exploit cluster-wide cache and compute resources.

4 NEBULA Overview

We present NEBULA, the first distributed LLM serving system to treat PIC segments as first-class citizens. Distributed PIC is more than introducing a new cache object: it entails joint, system-level optimization across view synchronization, cache management, and prefill scheduling. This section first presents the design principles (§4.1) and system architecture (§4.2), then outlines three new challenges posed by distributed PIC (§4.3). We then detail the three corresponding solutions: Optimistic, Event-Driven View Synchronization (§5), Shared, Self-Tuning Cache Management (§6), and Elastic, Locality-Aware Prefill Scheduling (§7).

4.1 Design Principles

Treat PIC segments as first-class citizens. Existing PIC works identify and reuse PIC segments only within individual workers, whereas existing distributed caching systems still manage prefix caches across nodes. Simply composing the two neither enables cross-node reuse of cached PIC segments nor allows independently computable PIC segments to be offloaded to idle workers (§3.2). A PIC segment must therefore be more than a piece of KV state internal to the inference engine. It must become a first-class distributed cache object and the common unit of operation for view synchronization, cache management, and prefill scheduling. Each PIC segment can then be independently indexed, placed, transferred, reused, or computed across the cluster, extending PIC from a worker-local optimization to a cluster-wide capability.

Complement prefix caching rather than replace it. The relative benefits of PC and PIC vary across workloads (§3.1), while rebuilding prefix cache with recomputation-based PIC still incurs overhead [6, 10, 31, 34]. A distributed PIC framework should therefore preserve existing prefix-caching semantics and add PIC as a complementary capability. The two cache types should share the same storage and transfer infrastructure while retaining their distinct matching semantics.

Make PIC algorithms pluggable. Different PIC algorithms handle segment reuse and recomputation in different ways (§2.2). A distributed PIC framework should therefore not be tied to a single algorithm. Instead, it should integrate existing PIC algorithms as pluggable backends through a unified interface that decouples cluster-wide cache management and prefill scheduling from algorithm-specific KV processing, enabling single-node PIC algorithms to operate in distributed settings.

NEBULA leaves KV reuse and recomputation policies, including their accuracy tradeoffs, to the PIC backend. It adapts different PIC backends for distributed execution and focuses on cluster-wide resource management.

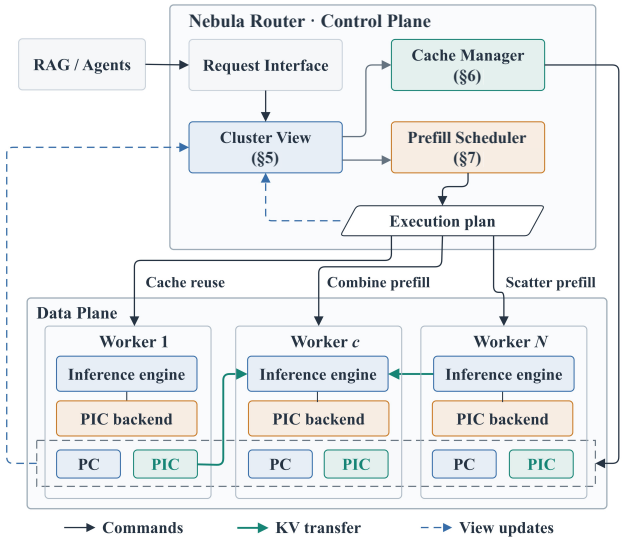


Figure 3: NEBULA system architecture.

RAG application input
System: You are a customer-support assistant.
User: <pic>A damaged product may be returned within 30 days.</pic>
<pic>Refunds use the original payment method.</pic>
Question: How long do I have to return a damaged product?

Agent application input
System, user, assistant: coding instructions and conversation history
Tool: <pic>[parser.py] `def parse_config(path): ...`</pic>
Tool: <pic>[SKILL.md] Normalize keys before validation.</pic>
User: Update the parser to follow these guidelines.

Figure 4: Application-defined text segmentation.

4.2 System Architecture

Framework components. As shown in Figure 3, NEBULA consists of a control plane and a data plane. The control plane makes cluster-wide cache and compute decisions, and the data plane executes them.

The control plane is implemented by a cluster-level router and contains three components:

- **Cluster View** maintains cluster-wide cache state and load state, providing a unified view for cache management and prefill scheduling (§5).
- **Cache Manager** uses this view to allocate shared cache capacity between PC and PIC (§6).
- **Prefill Scheduler** uses the same view to construct execution plans (§7).

The data plane comprises two PIC-enhanced components:

- **Cache Pool** retains existing prefix cache storage and transfer capabilities and extends them to PIC segments.
- **Inference Workers** retain their existing model execution paths and invoke specific PIC algorithms through a unified backend interface.

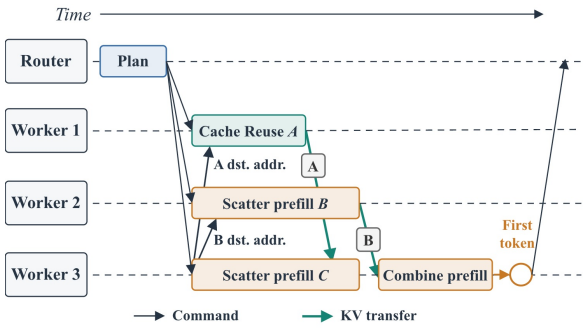


Figure 5: Execution timeline for one PIC request.

Application Interface and PIC Segments. NEBULA lets applications define where PIC is used. The application partitions each request into text segments: content enclosed by the PIC tags `<pic>...</pic>` forms an application-defined PIC segment, while each maximal unmarked span forms a regular segment. The interface removes these tags before tokenization. Unlike PIC segments, regular segments cannot be independently prefilled, cached, transferred, or reused across the cluster. Instead, they are reused through the prefix cache or recomputed in the request context on a prefix-cache miss.

Figure 4 shows example segmentations for RAG and agent requests. RAG applications mark retrieved documents as PIC segments in their request templates, while agent applications mark tool outputs, environment observations, and cross-agent messages as PIC segments. The remaining unmarked content forms regular segments. System prompts typically form stable request prefixes and can be reused through prefix caching, so they remain unmarked in these examples.

Request lifecycle. When a request arrives, the control plane first performs prefix matching over the complete request, then position-independent matching for PIC segments not covered by the prefix hit. The Prefill Scheduler uses the matching results and worker load to construct an execution plan. The plan specifies a worker for each PIC segment, either to supply cached KV or to perform prefill, and a *combine home*, where the required KV is gathered to complete the request's prefill. We detail plan generation in §7.

Figure 5 illustrates execution through three types of operations. **Scatter prefill** independently computes cold PIC segments on assigned workers and transfers their generated KV to the combine home when needed. **Cache reuse** makes matched PC/PIC KV available at the combine home, transferring remotely cached KV when needed. **Combine prefill** invokes the PIC backend at the combine home to reuse PIC segments, compute the remaining content, and perform necessary recomputation. Scatter workers can begin computation once they receive their tasks, but transfer generated KV via direct GPU writes only after receiving destination GPU addresses from the combine home. Scatter prefill, cache reuse, and control message delivery can overlap, while combine

prefill begins only after all required KV is available at the combine home.

Once prefill completes, NEBULA hands off the request and its KV state to a decode worker through the standard PD-disaggregated serving path. NEBULA leaves the decode path unchanged, focusing on reducing TTFT through prefill optimization.

4.3 Challenges

The framework above extends PIC from a single worker to the entire cluster. However, making the system efficient remains non-trivial: distributed PIC introduces three new system challenges in view synchronization, cache management, and prefill scheduling.

C1: Keeping the Cluster View Timely and Truthful under Fine-Grained Reuse. Distributed PIC substantially increases the number of independently scheduled units whose cache and compute states change asynchronously across workers, while routing decisions and actual execution are separated in time. An inaccurate view can cause invalid cache hits, whereas delayed updates can lead to redundant computation. The cluster view must therefore be both timely and truthful to support cache management and prefill scheduling (§5).

C2: Balancing PC and PIC Allocation under a Shared Budget. PC and PIC share finite cache capacity, but their relative reuse demand varies across workloads. Uncontrolled competition between them can crowd out frequently reused content, while fixed partitions can leave capacity idle for one type even as the other runs out of space. Cache management must therefore dynamically coordinate PC and PIC allocation as cluster-wide reuse demand changes (§6).

C3: Balancing Compute and Transfer under Shifting Load. Remote cache reuse incurs network transfer delays, while parallel prefill must wait for the slowest computation to finish. Under changing load, placing computation on cache-affine nodes may increase queueing delays, while placing it on less-loaded nodes may increase KV transfer costs. Scheduling must therefore balance compute and transfer when jointly selecting compute placement and parallelism (§7).

5 Optimistic, Event-Driven View Synchronization

5.1 Problem Analysis

Distributed PIC divides each request into multiple PIC segments that can be independently placed, transferred, and computed. This shift from request-level to segment-level scheduling increases the number of scheduling units by 11–106× at the median and up to 413× at P99 (Table 2). Dispatched segment computations may still be queued or running when

	Segments/request			Length (tokens)		
Workload	P50	P95	P99	P50	P95	P99
ChatRAG-doc2dial	106	112	113	334	670	1,190
ChatRAG-QuAC	35	127	203	536	773	939
RAGPulse	11	13	15	319	521	521
nebius-SWE-agent	31	253	413	69	1,032	1,431
τ-bench-Retail	14	34	44	93	1,240	1,240
ChatDev	22	65	73	818	2,253	2,780

Table 2: Distributions of segment counts and lengths across RAG and agent workloads.

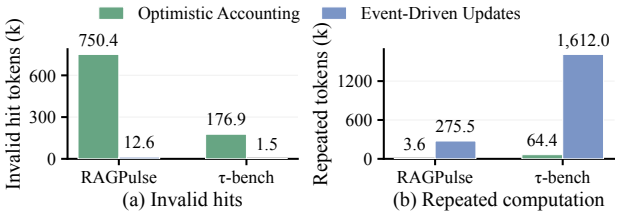


Figure 6: Limitations of Optimistic Accounting and Event-Driven Updates used in isolation: (a) invalid hit tokens and (b) repeated computation tokens on RAGPulse and τ-bench-Retail.

subsequent requests arrive, while cache insertions and evictions occur asynchronously across workers.

We construct two strawmen for distributed-PIC cache-directory maintenance based on existing view-synchronization mechanisms: (i) **Optimistic Accounting** infers PIC cache state from scheduling decisions, worker capacity, and an LRU model, following the SGLang router [24] and vLLM router [25]. (ii) **Event-Driven Updates** maintains the PIC cache index using worker-reported cache events, following Dynamo [18] and llm-d [15]. However, the optimistic strawman exposes assigned work immediately but can report invalid cache hits when its predicted PIC-segment locations drift from actual residency. The event-driven strawman does not assume cache availability before worker confirmation, which can lead to redundant prefill of the same segment across requests.

We compare the two variants using Qwen3.5-35B-A3B on a 7P1D cluster, replaying the RAGPulse and τ-bench-Retail traces at 40 QPS. We measure *invalid hit tokens*, where expected holders have neither usable KV nor earlier in-flight work, and *repeated computation tokens*, where directory misses trigger redundant segment prefill despite earlier in-flight work. Figure 6 shows that Optimistic Accounting incurs 59.4× and 114.3× as many invalid hit tokens as Event-Driven Updates on the two workloads, respectively. Conversely, Event-Driven Updates incurs 77.3× and 25.0× as many repeated computation tokens.

Therefore, under fine-grained PIC reuse, view synchronization cannot rely on router-side speculation or worker-side confirmation alone. It must address both view drift and delayed

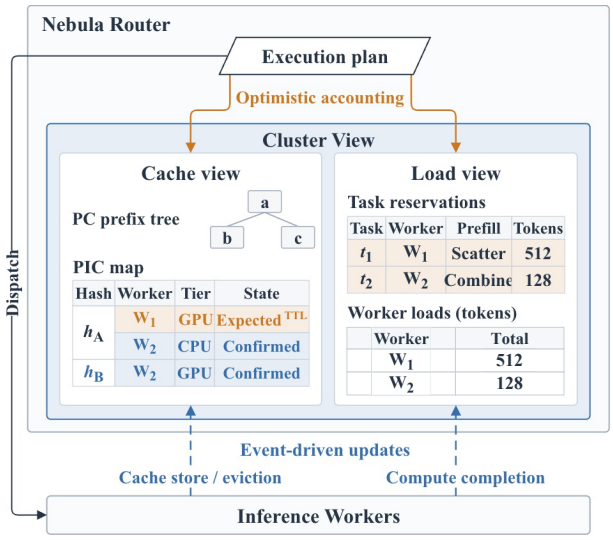


Figure 7: View synchronization in NEBULA.

updates, providing a basis that is both timely and truthful for cache management and prefill scheduling.

5.2 Our Solution

The key insight of NEBULA is to combine the router's immediate scheduling decisions with workers' execution feedback.

View Organization. The cluster view of NEBULA consists of a cache view and a load view (Figure 7). The cache view retains the existing PC prefix index and organizes the PIC map by content hash. For each PIC segment, it records its worker locations, storage tiers, and whether each record has been confirmed by a worker-side event. The load view records each task's target worker and reserved compute work and aggregates these reservations into each worker's outstanding compute load.

Optimistic Accounting and Event-Driven Updates. When dispatching an execution plan, the router immediately records expected PIC-segment locations in the cache directory and reserves compute load for scatter prefill and combine prefill tasks. Subsequent requests can thus identify and await matching in-flight results on the assigned workers, avoiding redundant prefill. During execution, workers send incremental events to the router: cache-store events confirm the corresponding locations, cache-eviction events remove invalid locations, and compute-completion reports release the corresponding tasks' load reservations.

NEBULA assigns each unconfirmed cache-location entry a TTL and removes it upon expiration. When a cache-store event arrives, the corresponding location becomes confirmed and is no longer subject to the TTL. Late cache-store events can still re-register the corresponding locations.

Unlike unconfirmed cache-location entries, load reservations do not expire on timeout, because their tasks may still be queued or running. The router releases a reservation only after confirming that the task has completed or will not execute, and deduplicates completion reports so that each reservation is released at most once.

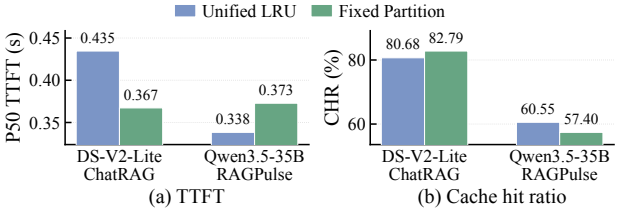


Figure 8: Cache allocation trade-off: (a) median TTFT and (b) token-weighted cache hit ratio.

6 Shared, Self-Tuning Cache Management

6.1 Problem Analysis

Recompute-based PIC methods may store different KV representations of the same segment in PC and PIC, creating competing demands for finite cache capacity. The relative demand for the two cache types depends on workload reuse patterns (§3.1)— doc2dial and QuAC reuse RAG documents under different prefixes, whereas nebius-SWE-agent and τ-bench-Retail repeatedly extend the same prefix; RAGPulse and ChatDev exhibit both patterns.

We construct two strawmen for PC–PIC cache allocation based on existing capacity-management policies: (i) **Unified LRU**, as used by MiniPIC [20], manages PC and PIC in a shared cache pool on each worker without cache-type quotas and selects eviction victims by access recency. (ii) **Fixed Partition**, following HYPIC's separate capacity budgets and per-pool LRU [10], divides capacity between PC and PIC at a preset ratio, with no borrowing between the two pools. However, Unified LRU allocates shared capacity solely through recency-based eviction, which may allow one cache type to crowd out frequently reused content from the other. Fixed Partition limits this interference but cannot adapt to changing demand, potentially leaving capacity idle in one pool while the other repeatedly evicts entries for lack of space.

We compare the two policies on a 7P1D cluster using DeepSeek-V2-Lite with ChatRAG-QuAC at 16 QPS and Qwen3.5-35B-A3B with RAGPulse at 40 QPS. Figure 8 shows that Fixed Partition achieves a higher cache hit ratio on ChatRAG (82.79% versus 80.68%), while Unified LRU wins on RAGPulse (60.55% versus 57.40%). Neither policy dominates across these two configurations.

Therefore, under a shared cache budget, the Cache Manager cannot rely solely on a unified eviction policy or fixed quotas to allocate capacity. Instead, it must self-tune the PC and PIC capacity shares as cluster-wide reuse demand changes, improving cache reuse under finite capacity.

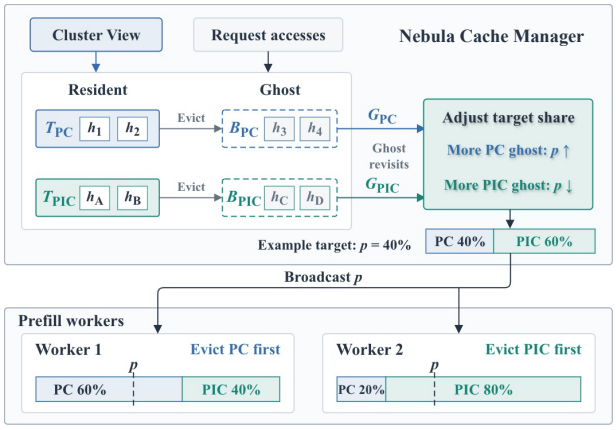


Figure 9: Self-tuning cache management in NEBULA.

6.2 Our Solution

The key insight of NEBULA is to use cluster-wide reuse feedback to dynamically adjust the shares of PC and PIC.

Cluster-Wide Reuse Feedback. Inspired by ARC's adaptation between recency and frequency [16], NEBULA uses revisits to evicted cache entries to identify which cache type needs more space. As shown in Figure 9, the router maintains four queues: T_{PC} and T_{PIC} track PC and PIC entries present in the cluster, while B_{PC} and B_{PIC} track previously cached entries no longer present in the cluster. All four queues store only identifiers, not cached data.

When an entry appears in the cluster view, the router adds it to the corresponding T queue and removes its history from the corresponding B queue. When the cluster view no longer reports any available holder for an entry, the router moves it from T to B.

Reuse feedback follows the prefix-first principle (§4.1). The router first matches existing prefix caches in T_{PC}, then checks B_{PC} to retain revisit feedback for missing prefix caches, even when PIC can still serve the same content. It then checks T_{PIC} and B_{PIC} in order to assess PIC reuse demand. Each token in a request contributes to at most one feedback category.

Self-Tuning Capacity Shares. Let G_{PC} and G_{PIC} denote the token counts covered by ghost revisits for the two cache types. Let $p \in [0,1]$ be the global target soft share for PC, with $1-p$ for PIC. The router updates this share over request windows:

$$g = \frac{G_{\text{PC}} - G_{\text{PIC}}}{G_{\text{PC}} + G_{\text{PIC}}}, \qquad p_{\text{new}} = \text{clip}_{[0,1]}\left[p + 0.01g\right]. \tag{1}$$

More PC ghost tokens increase p, while more PIC ghost tokens decrease it.

NEBULA initializes p to 0.5 and updates it every $U = 16$ requests, changing it by at most one percentage point per update. After each update, the router asynchronously broadcasts the new allocation ratio to all prefill workers.

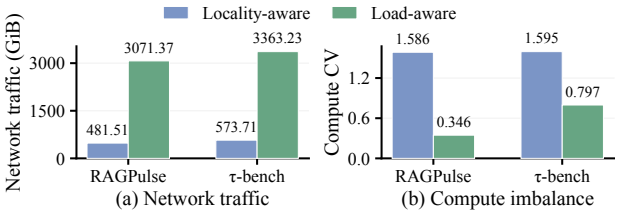


Figure 10: Locality–load trade-off on RAGPulse and τ-bench-Retail. Locality-aware reduces network traffic, while Load-aware lowers the CV of cumulative prefill tokens across workers.

Worker-Side Soft Allocation. All prefill workers use the same target ratio. Let R_{PC} and R_{PIC} denote the local cache occupancy of PC and PIC, respectively. If

$$R_{\text{PC}} > p\left(R_{\text{PC}} + R_{\text{PIC}}\right), \tag{2}$$

the worker prioritizes PC eviction; otherwise, it prioritizes PIC eviction. Updating p changes only the cache-type priority for subsequent reclamation and triggers no immediate eviction.

For tiered storage, NEBULA maintains the four queues separately for each storage tier and adjusts its PC/PIC target shares using that tier's reuse feedback. Workers independently apply soft allocation at each tier based on its target shares.

7 Elastic, Locality-Aware Prefill Scheduling

7.1 Problem Analysis

Distributed PIC allows cold PIC segments to be prefilled in parallel across workers, while PC and PIC may reside on different workers. Under changing cluster load, scheduling must jointly select each cache's reuse path, the combine home and the placement of cold PIC-segment computation so that the required KV becomes ready at the combine home as early as possible for the subsequent combine prefill.

Building on existing cache-aware and load-balancing approaches, we construct two strawman schedulers for distributed PIC: (i) **Locality-aware** selects the combine home with the most local PC/PIC hits, computes cold PIC segments there, and fetches the remaining PIC hits from other workers. (ii) **Load-aware** assigns scatter and combine prefill tasks to workers using longest-processing-time-first (LPT) scheduling and transfers both cached and generated KV from remote workers to the combine home. However, Locality-aware may increase queueing delays by placing computation on a cache-affine but busy worker. Load-aware may incur higher transfer costs when fetching cache hits or returning newly generated KV. Having more local cache hits or more balanced computation does not necessarily lead to earlier prefill completion.

We compare the two schemes using Qwen3.5-35B-A3B on a 7P1D cluster, replaying the RAGPulse and τ-bench-Retail traces at 8 QPS. We measure total network traffic and

Algorithm 1 Greedy Prefill Planning

Require: $\mathcal{S}$, $\mathcal{W}$, $\{\mathcal{H}_w, Q_w\}$, $\{P_i\}$, cache-hit sources, $\widehat{T}(\cdot)$
Ensure: Execution plan π_{best}
1: $cold \leftarrow$ sort $\mathcal{S} \setminus \bigcup_{w\in\mathcal{W}} \mathcal{H}_w$ by decreasing P_i
2: $\Pi \leftarrow \varnothing$
3: **for** each $c \in \mathcal{W}$ **do**
4: $\quad a \leftarrow$ empty mapping
5: $\quad$ **for** each $i \in \mathcal{S}$ **do**
6: $\quad\quad a(i) \leftarrow c$ if $i \in cold$, else its cache source
7: $\quad \pi \leftarrow (c, a)$
8: $\quad$ **for** each $i \in cold$ **do**
9: $\quad\quad$ **if** $\mathcal{W} \setminus \{c\} = \varnothing$ **then break**
10: $\quad\quad w \leftarrow \arg\min_{v\in\mathcal{W}\setminus\{c\}} [Q_v + P_v(\pi)]$
11: $\quad\quad a' \leftarrow \text{copy}(a)$
12: $\quad\quad a'(i) \leftarrow w$
13: $\quad\quad \pi' \leftarrow (c, a')$
14: $\quad\quad$ **if** $\widehat{T}(\pi') < \widehat{T}(\pi)$ **then**
15: $\quad\quad\quad a \leftarrow a'$
16: $\quad\quad\quad \pi \leftarrow \pi'$
17: $\quad \Pi \leftarrow \Pi \cup \{\pi\}$
18: $\pi_{\text{best}} \sim \text{Uniform}\left(\arg\min_{\pi\in\Pi} \widehat{T}(\pi)\right)$
19: **return** π_{best}

quantify compute imbalance using the population coefficient of variation (CV) of cumulative prefill tokens across prefill workers. As Figure 10 shows, Load-aware transfers 6.4× and 5.9× as much data as Locality-aware on the two workloads, respectively, but reduces compute CV by 4.6× and 2.0×.

Therefore, under changing request load and cache distribution, prefill scheduling cannot prioritize cache locality or load balance alone. Instead, it must jointly consider compute load and KV transfer costs when selecting placement and parallelism to minimize prefill completion time.

7.2 Our Solution

The key insight of NEBULA is to jointly account for queueing, computation, and KV transfer when estimating a distributed PIC request's prefill completion time.

Estimating Prefill Completion Time. NEBULA constructs execution plans in request arrival order (FIFO). Each plan consists of a combine home c and a mapping a from the request's PIC segments to workers:

$$\pi = (c, a), \qquad a : \mathcal{S} \to \mathcal{W}, \quad c \in \mathcal{W}, \tag{3}$$

where $\mathcal{S}$ is the request's PIC segment set and $\mathcal{W}$ is the cluster's worker set.

To estimate the completion time of plan π, NEBULA uses the cache and load state in the cluster view. Here, $\mathcal{H}_w$ denotes the set of segments cached at worker w, and Q_w is its total outstanding compute load, including scatter and combine prefill. Q_w is measured in tokens and excludes the current request. Let $\mathcal{S}_w(\pi) = \{i \in \mathcal{S} \mid a(i) = w\}$ denote the segments assigned to worker w. For cache hits $\mathcal{S}_w(\pi) \cap \mathcal{H}_w$, NEBULA reuses the KV already stored at that worker; for cache misses $\mathcal{S}_w(\pi) \setminus \mathcal{H}_w$, it performs scatter prefill there.

We estimate the prefill completion time of plan π as:

$$\widehat{T}(\pi) = \max_{w\in\mathcal{W}} [Q_w(\pi) + P_w(\pi) + D_w(w \to c; \pi)] + C. \tag{4}$$

The estimate sums queueing $Q_w(\pi)$, computation $P_w(\pi)$, and transfer time $D_w(w \to c; \pi)$ at each worker, takes the maximum across workers as the estimated KV-ready time, and adds the request's combine prefill work C. In this model, matched prefix KV is also represented as cached segments and included in the transfer cost. All terms are normalized to equivalent prefill tokens:

$$\begin{aligned} Q_w(\pi) &= \begin{cases} Q_w, & \mathcal{S}_w(\pi) \setminus \mathcal{H}_w \neq \varnothing, \\ 0, & \text{otherwise}, \end{cases} \\ P_w(\pi) &= \sum_{i\in\mathcal{S}_w(\pi)\setminus\mathcal{H}_w} P_i, \\ D_w(w \to c; \pi) &= d(w \to c) \sum_{i\in\mathcal{S}_w(\pi)} P_i. \end{aligned} \tag{5}$$

Here, P_i is the token count of segment i. The per-worker queueing and scatter-computation terms arise only when a worker is assigned a cache miss. The transfer term covers both cache hits and newly computed KV. The coefficient $d(w \to c)$ is obtained through offline profiling: it is the per-token KV transfer time from worker w to worker c, normalized by the per-token prefill time, with $d(c \to c) = 0$. The request's combine prefill workload C, which includes computation of the remaining content and necessary PIC recomputation, is constant across candidate plans. Combine prefill receives the highest scheduling priority once its required KV is ready, so the estimate omits its queueing delay.

Greedy Plan Generation. Using this prefill completion time estimate, NEBULA greedily optimizes the execution plan to minimize completion time (Algorithm 1).

The algorithm first enumerates available combine homes c and constructs an initial plan π for each: cache hits are assigned to the source workers, and all cold PIC segments are assigned to c (lines 3–7). This initial assignment preserves cluster-wide cache reuse while avoiding KV return traffic from cold PIC-segment computation.

For each cold PIC segment in decreasing order of length, the algorithm constructs a candidate plan π' by reassigning it to the worker $w \neq c$ minimizing $Q_w + P_w(\pi)$ (lines 8–13). The search holds Q_w fixed, while updating $P_w(\pi)$ after each accepted move. Load guides candidate selection, but a move is accepted only if $\widehat{T}(\pi') < \widehat{T}(\pi)$ (lines 14–16). Otherwise, the plan remains unchanged and the search proceeds to the next segment.

After searching all candidate combine homes, it returns a plan with the lowest estimated completion time (lines 17–19).

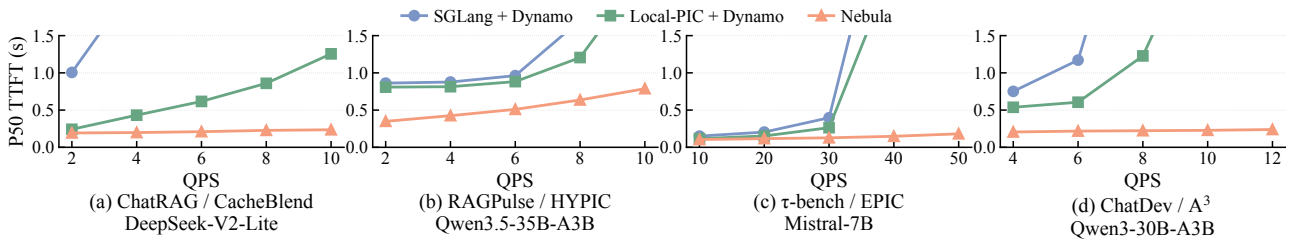


Figure 11: End-to-end median TTFT versus QPS: (a) ChatRAG with DeepSeek-V2-Lite and CacheBlend, (b) RAGPulse with Qwen3.5-35B-A3B and HYPIC, (c) τ-bench with Mistral-7B and EPIC, and (d) ChatDev with Qwen3-30B-A3B and A^3.

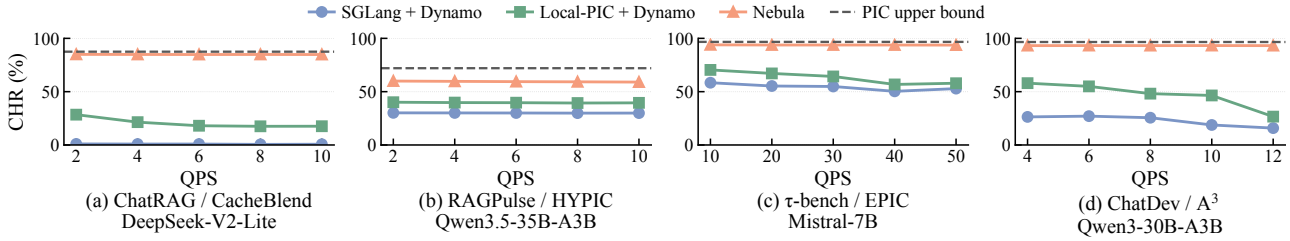


Figure 12: End-to-end cache hit ratio versus QPS, with the same configurations as Figure 11.

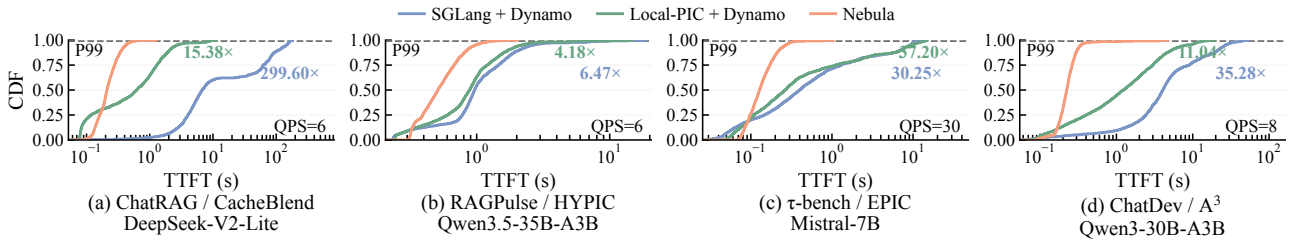


Figure 13: End-to-end TTFT CDFs, with the same configurations as Figure 11.

8 Implementation

We implement NEBULA on SGLang [33] and Dynamo [18], reusing SGLang's radix tree and tiered storage infrastructure and Dynamo's cluster orchestration. On the workers, we extend SGLang's forward execution path to support CacheBlend [31], A^3 [34], EPIC [6] and HYPIC [10], and adapt its storage pool allocation and indexing to manage PIC segments across storage tiers. On the router, we extend Dynamo's cluster directory to support cache routing and prefill scheduling for distributed PIC (§4.2). We use NIXL [19] for efficient point-to-point transfers of PIC between workers across storage tiers. Our implementation comprises approximately 19K lines of code.

9 Evaluation

9.1 Setup

Hardware. We run our experiments on two identical servers, each equipped with four NVLink-connected NVIDIA H20-3e GPUs, two 48-core Intel Xeon Platinum 8575C CPUs, 1 TiB of host memory, and four 200 Gbps RDMA-capable Ethernet ports.

Models. We evaluate four models: DeepSeek-V2-Lite [3], Qwen3.5-35B-A3B [23], Mistral-7B [17], and Qwen3-30B-A3B [30]. All models use seven prefill workers and one decode worker (7P1D), with one GPU per worker (TP1). We run DeepSeek-V2-Lite with CacheBlend, Qwen3.5-35B-A3B with HYPIC, Mistral-7B with EPIC, and Qwen3-30B-A3B with A^3.

Workloads. We use four public RAG and agent workloads: ChatRAG-QuAC [2, 12] for conversational RAG, RAGPulse [28] for RAG serving, τ-bench-Retail [32] for tool-using agents, and ChatDev [21] for multi-agent software development. Table 1 summarizes the theoretical cache-reuse breakdown for these workloads, while Table 2 reports the distributions of segment counts and lengths. We preserve the original request order and replay each trace with Poisson arrivals at varying rates. Since PIC targets TTFT reduction, we limit each request to one output token while retaining one decode worker to preserve prefill–decode disaggregation deployment.

Methods. We compare three configurations: (i) **SGLang + Dynamo** combines SGLang [33] workers with Dynamo's prefix-cache-aware router [18], representing a common pro-

duction deployment; (ii) **Local-PIC + Dynamo** is the direct-composition strawman described in §3.2, deploying PIC-enabled workers behind Dynamo without remote cache reuse or parallel prefill of PIC segments; and (iii) **NEBULA** integrates our three system designs to support remote PIC reuse and parallel prefill. Component experiments vary the mechanism under study while holding the remaining configuration fixed. Unless otherwise specified, each configuration runs from a fresh cluster without warmup.

9.2 End-to-end performance

We evaluate NEBULA's end-to-end performance using the four real-world workloads and four models described in §9.1.

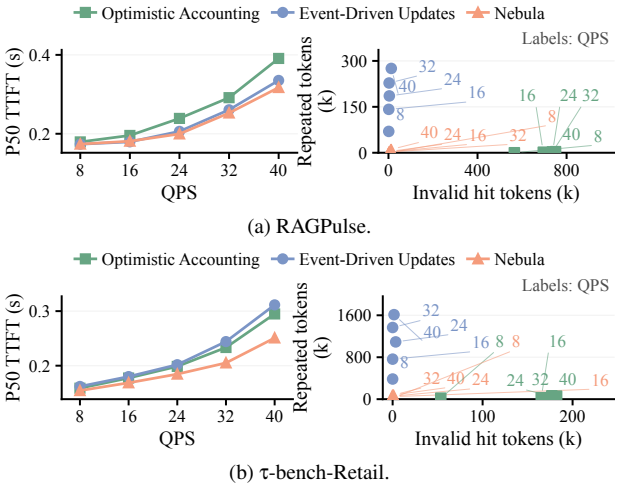
Median and tail TTFT. Figure 11 shows that NEBULA achieves the lowest median TTFT across all measured settings. Across these settings, NEBULA reduces median TTFT by an average of 61.4% and 76.7% relative to Local-PIC + Dynamo and SGLang + Dynamo, respectively. At the highest tested rate for each workload, its median TTFT is 0.24, 0.79, 0.18, and 0.24 s for ChatRAG, RAGPulse, τ-bench-Retail, and ChatDev, respectively, compared with 1.26, 2.39, 3.64, and 9.57 s for Local-PIC + Dynamo and 75.73, 2.71, 8.79, and 8.46 s for SGLang + Dynamo. The results demonstrate the end-to-end benefits of jointly supporting remote cache reuse and parallel prefill beyond worker-local PIC alone.

We also report the TTFT CDF in Figure 13 at 6, 6, 30, and 8 QPS for ChatRAG, RAGPulse, τ-bench-Retail, and ChatDev, respectively. At P99, NEBULA reduces TTFT by 93.5%, 76.1%, 97.3%, and 90.9% relative to Local-PIC + Dynamo on these workloads, respectively. Relative to SGLang + Dynamo, the reductions range from 84.6% to 99.7%.

TTFT-constrained goodput. We measure goodput as the number of successful requests meeting the TTFT SLO divided by the time from the first request's start to the last request's completion. With a 1 s TTFT SLO, NEBULA achieves 11.66 requests/s on ChatRAG at 12 QPS, compared with 1.68 requests/s for Local-PIC + Dynamo.

Cache hit ratio. We measure Cache Hit Ratio (CHR) as the total number of input tokens served from cache divided by the total number of input tokens. Figure 12 shows that NEBULA achieves the highest CHR across all measured settings. At the highest tested rates, its CHRs are 84.82%, 58.99%, 93.67%, and 93.17% for ChatRAG, RAGPulse, τ-bench-Retail, and ChatDev, respectively, exceeding Local-PIC + Dynamo by 19.48–67.23 percentage points.

Compared with SGLang + Dynamo, NEBULA recovers 84.7%, 42.1%, 86.3%, and 91.2% of the tokens missed by prefix caching on ChatRAG, RAGPulse, τ-bench-Retail, and ChatDev, respectively.



(a) RAGPulse.

(b) τ-bench-Retail.

Figure 14: View synchronization at different request rates. The left plots show median TTFT, while the right plots compare invalid hit and repeated computation token counts.

9.3 View Synchronization

Invalid hits and repeated computation. We compare NEBULA with Optimistic Accounting and Event-Driven Updates described in §5.1, varying only the PIC directory maintenance mechanism. Using Qwen3.5-35B-A3B, we replay the RAGPulse and τ-bench-Retail traces at different request rates. NEBULA uses a 5 s TTL for unconfirmed entries.

Figure 14 shows that NEBULA achieves the lowest median TTFT on both workloads. At 40 QPS, it reduces median TTFT by 18.9% on RAGPulse and 14.9% on τ-bench-Retail relative to Optimistic Accounting. Relative to Event-Driven Updates, the reductions are 5.4% and 19.5%, respectively.

At 40 QPS, NEBULA reduces the number of invalid hit tokens by 98.4% on RAGPulse and 100% on τ-bench-Retail relative to Optimistic Accounting. Relative to Event-Driven Updates, it reduces the number of repeated computation tokens by 96.1% and 95.1%, respectively.

9.4 Cache Management

Cache capacity management variants. We compare NEBULA with Unified LRU and Fixed Partition from §6.1, holding view synchronization and prefill scheduling fixed. We run Qwen3.5-35B-A3B on RAGPulse and Qwen3-30B-A3B on ChatDev at different request rates, both on a 7P1D cluster with TP1. Fixed Partition allocates half of the cache capacity to PC and half to PIC. In this experiment, we use the first 800 requests as warmup and report metrics over the remaining requests.

Figure 15 shows that NEBULA achieves the lowest TTFT and highest CHR. On RAGPulse at 48 QPS, NEBULA increases CHR by 4.32 and 4.88 percentage points and reduces median TTFT by 11.5% and 14.7% relative to Unified LRU

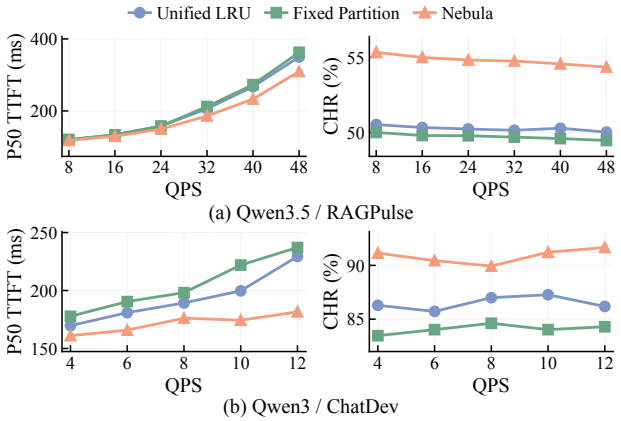


Figure 15: Cache management policies versus QPS for (a) Qwen3.5-35B-A3B on RAGPulse and (b) Qwen3-30B-A3B on ChatDev.

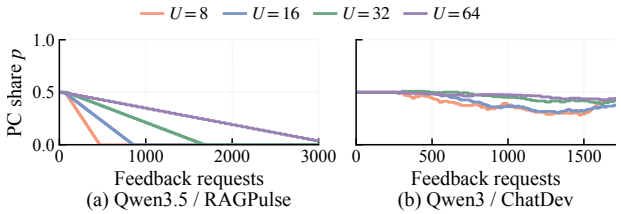


Figure 16: Trajectory of the PC target soft share p under different update windows U for (a) Qwen3.5-35B-A3B on RAGPulse and (b) Qwen3-30B-A3B on ChatDev.

and Fixed Partition, respectively. On ChatDev at 12 QPS, the corresponding CHR gains are 5.48 and 7.37 percentage points, with median TTFT reductions of 20.8% and 23.4%.

Update-window sensitivity and capacity-share trajectory. We next study the update-window sensitivity and how the update window affects the shares of PC and PIC. We run Qwen3.5-35B-A3B on RAGPulse at 12 QPS and Qwen3-30B-A3B on ChatDev at 6 QPS, varying the update window U over 8, 16, 32, and 64 requests.

Figure 16 shows how the PC target share p evolves under different update windows. On RAGPulse, p approaches zero at the maximum adjustment rate, consistent with its preference for PIC and few PC hits. On ChatDev, p decreases and then rises as feedback changes, indicating that the relative demand for PC and PIC varies over time.

Figure 17 shows that $U = 8$ gives the lowest measured median TTFT on RAGPulse (121.46 ms), while $U = 32$ gives the lowest on ChatDev (146.94 ms). The $U = 16$ results are 122.33 ms and 147.32 ms, respectively, achieving median TTFT within 1 ms of the best tested window on both workloads. Considering the update frequency as well, we use $U = 16$ by default.

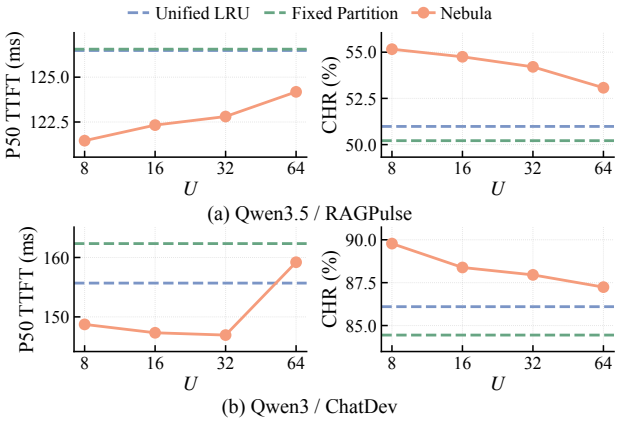


Figure 17: Effect of the update window U: median TTFT and CHR for (a) Qwen3.5-35B-A3B on RAGPulse and (b) Qwen3-30B-A3B on ChatDev. Dashed lines show the two baselines, which do not depend on U.

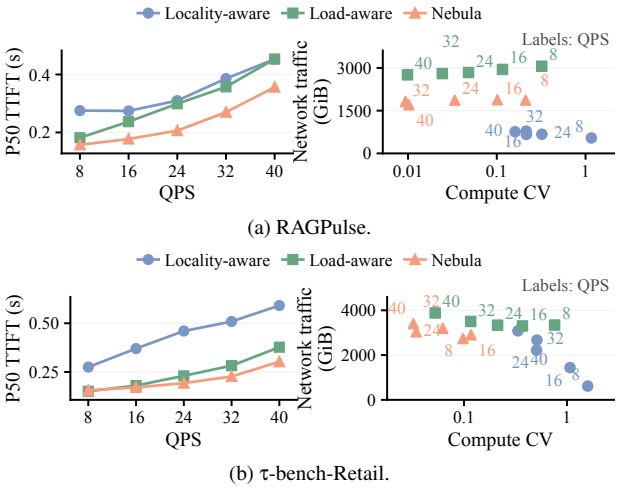


Figure 18: Prefill scheduling at different request rates. The left plots show median TTFT, while the right plots compare total NIXL TX payload and compute CV.

9.5 Prefill Scheduling

Traffic and Load Balance. We compare NEBULA with the Locality-aware and Load-aware strawmen from §7.1. We deploy Qwen3.5-35B-A3B and replay the RAGPulse and τ-bench-Retail traces at different request rates. All schedulers support remote PIC reuse.

Figure 18 shows that NEBULA achieves the lowest median TTFT on both workloads at 24–40 QPS. At 40 QPS, it reduces median TTFT by 21.4% and 48.9% relative to Locality-aware, and by 21.4% and 19.8% relative to Load-aware, on RAGPulse and τ-bench-Retail, respectively.

We measure total NIXL TX payload and the population CV of cumulative prefill tokens across prefill workers. At 40 QPS, NEBULA reduces traffic by 38.1% and 12.3% relative to Load-aware, and compute CV by 93.6% and 90.3% relative

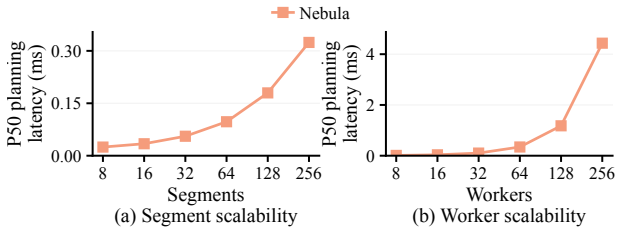


Figure 19: Scheduling overhead of NEBULA. (a) Varying the number of segments with 16 workers. (b) Varying the number of workers with 16 segments.

to Locality-aware, on RAGPulse and τ-bench-Retail, respectively. At this rate, Locality-aware transfers less data than NEBULA but has higher compute CV on both workloads.

Scheduling overhead. We measure the latency of greedy plan generation (§7.2) on one pinned vCPU. We uniformly sample segment lengths from 128–2,048 tokens and outstanding worker loads from 0–8,192 tokens, with a 50% segment cache hit ratio and each cache hit placed on a uniformly selected worker. For each setting, we measure 1,024 planning calls after 64 warmup calls. Figure 19 shows that median planning latency is 0.32 ms with 256 segments and 16 workers, and 4.43 ms with 16 segments and 256 workers.

10 Conclusion

NEBULA is the first distributed LLM serving framework to treat PIC segments as first-class citizens, enabling remote cache reuse and parallel prefill. It combines optimistic accounting with event-driven updates to maintain a timely and truthful cluster view, adapts PC/PIC capacity shares using cluster-wide ghost feedback, and selects prefill placement and parallelism through a unified completion-time estimate. Across four models and four real-world RAG and agentic workloads, NEBULA reduces median TTFT by an average of 61.4% compared with the strongest existing baseline and recovers an average of 76.1% of the tokens missed by prefix caching.

References

[1] Ibragim Badertdinov, Maria Trofimova, Yury Anapolskiy, Sergey Abramov, Karina Zainullina, Alexander Golubev, Sergey Polezhaev, Daria Litvintseva, Simon Karasik, Filipp Fisin, Sergey Skvortsov, Maxim Nekrashevich, Anton Shevtsov, and Boris Yangel. Scaling data collection for training software engineering agents. *Nebius Blog*, 2024. https://nebius.com/blog/posts/scaling-data-collection-for-training-swe-agents.

[2] Eunsol Choi, He He, Mohit Iyyer, Mark Yatskar, Wentau Yih, Yejin Choi, Percy Liang, and Luke Zettlemoyer. QuAC: Question answering in context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2174–2184. Association for Computational Linguistics, 2018.

[3] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[4] Song Feng, Hui Wan, Chulaka Gunasekara, Siva Patel, Sachindra Joshi, and Luis Lastras. doc2dial: A goal-oriented document-grounded dialogue dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8118–8128. Association for Computational Linguistics, 2020.

[5] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. MemServe: Context caching for disaggregated LLM serving with elastic memory pool. arXiv preprint arXiv:2406.17565, 2024.

[6] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Zhang Qin, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 24391–24402. PMLR, 2025.

[7] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, SOSP '23, pages 611–626. ACM, 2023.

[9] Yang Liu, Yunfei Gu, Liqiang Zhang, Chentao Wu, Guangtao Xue, Jie Li, Minyi Guo, Junhao Hu, and Jie Meng. CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies*, FAST '26, pages 83–99. USENIX Association, 2026.

[10] Yifei Liu, Juntong Wu, Yang Liu, Junhao Hu, Minghao Li, Xiaoxu Chen, and Weihang Chen. HYPIC: Ac-

celerating hybrid-attention LLM serving with position-independent caching. arXiv preprint arXiv:2607.01299, 2026.

[11] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. arXiv preprint arXiv:2510.09665, 2025.

[12] Zihan Liu, Wei Ping, Rajarshi Roy, Peng Xu, Chankyu Lee, Mohammad Shoeybi, and Bryan Catanzaro. ChatQA: Surpassing GPT-4 on conversational QA and RAG. In *Advances in Neural Information Processing Systems*, volume 37, pages 15416–15459. Curran Associates, Inc., 2024.

[13] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6588–6601. Association for Computational Linguistics, 2025.

[14] Dongyang Ma, Yan Wang, and Tian Lan. Block-attention for efficient prefilling. In *The Thirteenth International Conference on Learning Representations*, pages 63774–63788, 2025.

[15] Sai Sindhur Malleni, Raúl Sevilla, Aleksei Vasilevskii, José Castillo Lema, and André Bauer. Evaluating Kubernetes performance for GenAI inference: From automatic speech recognition to LLM summarization. In *Proceedings of the 17th ACM/SPEC International Conference on Performance Engineering*, pages 135–146. Association for Computing Machinery, 2026.

[16] Nimrod Megiddo and Dharmendra S. Modha. ARC: A Self-Tuning, low overhead replacement cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*, San Francisco, CA, March 2003. USENIX Association.

[17] Mistral AI. Mistral-7B-Instruct-v0.2. Hugging Face model card. https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2, 2023.

[18] Tiyasa Mitra, Ritika Borkar, Nidhi Bhatia, Shivam Raj, Hongkuan Zhou, Yan Ru Pei, Vishwanath Venkatesan, Kyle Kranen, Ramon Matas, Dheevatsa Mudigere, Ritchie Zhao, Maximilian Golub, Arpan Dutta, Suresh Nambi, Sailaja Madduri, Dharmesh Jani, Brian Pharris, Itay Neeman, and Bita Darvish Rouhani. Beyond the buzz: A pragmatic take on inference disaggregation. In *Proceedings of Machine Learning and Systems*, volume 8, pages 2001–2015. MLSys, 2026.

[19] NVIDIA. NVIDIA inference xfer library (NIXL). https://github.com/ai-dynamo/nixl. Software repository; accessed: 2026-10-16.

[20] Nathan Ordonez and Thomas Parnell. MiniPIC: Flexible position-independent caching in <100LOC. arXiv preprint arXiv:2606.13126, 2026.

[21] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186. Association for Computational Linguistics, 2024.

[22] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies*, FAST '25, pages 155–170. USENIX Association, 2025.

[23] Qwen Team. Qwen3.5: Towards native multimodal agents. Qwen Technical Blog. https://qwen.ai/blog?id=qwen3.5, 2026.

[24] SGLang Project. SGLang Model Gateway: Cache-aware load balancing router. https://github.com/sgl-project/sglang/blob/main/sgl-model-gateway/src/policies/cache_aware.rs. Accessed: 2026-09-11.

[25] vLLM Project. vLLM Router: Load balancing policies. https://github.com/vllm-project/router/blob/e7989e20b0a733fbd4e9ee6637d2df13373e9d04/src/policies/cache_aware.rs. Source snapshot; accessed: 2026-10-16.

[26] Qian Wang, Zahra Yousefijamarani, Morgan Lindsay Heisler, Rongzhi Gu, Xiaolong Bai, Yizhou Shan, Wei Zhang, Lan Wang, Ying Xiong, Yong Zhang, and Zhenan Fan. MEPIC: Memory efficient position independent caching for LLM serving. arXiv preprint arXiv:2512.16822, 2025.

[27] Shihao Wang, Jiahao Chen, Yanqi Pan, Hao Huang, Yichen Hao, Xiangyu Zou, Wen Xia, Chongyang Qiu, Wentao Zhang, and Pengfei Wang. ProphetKV: User-query-driven selective recomputation for efficient KV cache reuse in retrieval-augmented generation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 125415–125434. PMLR, 2026.

[28] Zhengchao Wang, Yitao Hu, Jianing Ye, Zhuxuan Chang, Jiazheng Yu, Youpeng Deng, and Keqiu Li. RAGPulse: An open-source RAG workload trace to optimize RAG serving systems. arXiv preprint arXiv:2511.12979, 2025.

[29] Bingyang Wu, Zili Zhang, Yinmin Zhong, Guanzhe Huang, Yibo Zhu, Xuanzhe Liu, and Xin Jin. TokenLake: A unified segment-level prefix cache pool for fine-grained elastic long-context LLM serving. arXiv preprint arXiv:2508.17219, 2025.

[30] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

[31] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, pages 94–109. ACM, 2025.

[32] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, pages 9965–10017, 2025.

[33] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, pages 62557–62583, 2024.

[34] Yuechi Zhou, Yi Su, Jianxin Zhang, Juntao Li, Qingrong Xia, Zhefeng Wang, Xinyu Duan, and Baoxing Huai. A^3: Attention-aware accurate KV cache fusion for fast large language model serving. arXiv preprint arXiv:2511.17560, 2025. Accepted at EMNLP 2026; arXiv version cited.